

АКАДЕМИЈА ТЕХНИЧКО-УМЕТНИЧКИХ СТРУКОВНИХ СТУДИЈА
БЕОГРАД

ОДСЕК ВИСОКА ШКОЛА ЕЛЕКТРОТЕХНИКЕ И РАЧУНАРСТВА

Никола Раденковић

**Развој веб апликације за стримовање и репродукцију
музике**

- ПРОЈЕКТНИ ЗАДАТАК -



Београд, март 2026.

Кандидат: **Никола Раднековић**

Број индекса: **ИС-38/23**

Студијски програм: **Информациони системи**

Тема: **Развој веб апликације за стримовање и репродукцију музике**

Основни задаци:

- 1. Пројектовање веб апликације за стримовање музике.**
- 2. Развој базе података и корисничког интерфејса.**
- 3. Имплементација разних функционалности.**

(регистрација, пријава, додавање песама, креирање корисничких албума, слушање песама)

Предметни наставник:

Београд, март 2026 године.

др Габријела Димић, виши предавач ВИШЕР

РЕЗИМЕ:

Овај пројекат представља процес развоја веб апликације за стримовање и емитовање музике. Циљ пројекта је креирање функционалне платформе која омогућава корисницима да претражују, слушају и организују музички садржај путем интернета. Други циљ овог пројекта је стицање практичног искуства у развоју веб апликација и унапређење знања из области. Апликација је реализована коришћењем веб технологија као што су HTML, CSS и JavaScript за израду корисничког интерфејса, за серверску страну су коришћени PHP и MySQL.

Кључне речи: стримовање музике, веб апликација, емитовање

ABSTRACT:

This project represents the process of developing a web application for streaming and broadcasting music. The goal of the project is to create a functional platform that allows users to search, listen to and organize music content via the Internet. The second goal of this project is to gain practical experience in web application development and improve knowledge in the field. The application was implemented using web technologies such as HTML, CSS and JavaScript for creating the user interface, PHP and MySQL were used for the server side.

Key words: streaming music, web application, broadcasting

САДРЖАЈ:

1.	Дефинисање пословних процеса и сценарија функционисања система	1
1.1.	Анализа проблема и потреба тржишта	1
1.2.	Опис пословног процеса	1
1.3.	Сценарио функционисања	1
2.	Опис функционалности по улогама	2
2.1.	Непријављени корисник (гост)	2
2.2.	Пријављени корисник (слушалац)	2
2.3.	Администратор	2
2.4.	Матрица права приступа	2
3.	Кратак опис коришћених технологија	3
3.1.	Frontend: Кориснички интерфејс	3
3.2.	Backend: Серверска логика	3
3.3.	База података	3
3.4.	Развојни алати (Environment)	3
4.	Логотип	5
4.1.	Логотип	5
5.	Моделовање базе података	6
5.1.	Концептуални модел	6
5.2.	Логички модел	7
5.3.	Физички модел	8
6.	Приказ и опис делова кода	13
7.	Закључак	17
8.	Литература	18

1. ДЕФИНИСАЊЕ ПОСЛОВНИХ ПРОЦЕСА И СЦЕНАРИЈА ФУНКЦИОНИСАЊА СИСТЕМА

1.1. АНАЛИЗА ПРОБЛЕМА И ПОТРЕБА ТРЖИШТА

Данас сви слушамо музику преко стриминг сервиса, али проблем је што су велики системи (попут Spotify-а) прекомпликовани за мале, независне ауторе и локалне заједнице. Циљ мог пројекта је да направим једноставну веб платформу где админи могу брзо да окаче своје песме, а корисници да их лако пронађу и организују у своје листе, без сувисних опција и реклама.

1.2. ОПИС ПОСЛОВНОГ ПРОЦЕСА

Све креће од **аутентификације**. Гост мора да креира налог па да се улогује како би систем знао које су његове песме и плејлисте.

Админ бира аудио фајл са свог компјутера и попуњава основне податке (назив, аутор, жанр, фајл). РНР проверава формат, величину и друге параметре фајла и сачува га у посебан простор на серверу. Путању фајла чува у бази података.

Корисник (Слушалац) на почетној страни види све песме. Преко претраге проналази специфичне песме. Када кликне на “Play”, апликација не освежава целу страницу, већ плејер одмах повлачи песму са сервера и креће емитовање.

1.3. СЦЕНАРИО ФУНКЦИОНИСАЊА

Корисник се улогује и жели да направи своју плејлисту народних песми. Он упише име плејлисте и затим изабере опцију “**Napravi Playlistu**” и одмах се сачува у бази. Затим кроз **претрагу** тражи песме. Поред сваке песме има дугме за додавање у своју листу. На крају, корисник отвара своју плејлисту и пушта музику.

2. ОПИС ФУНКЦИОНАЛНОСТИ ПО УЛОГАМА

2.1. НЕПРИЈАВЉЕНИ КОРИСНИК (ГОСТ)

Гост може да види само логин/регистер страницу. Покушај уласка на било коју другу страницу ће га аутоматски пребацити на логин/регистер страницу. Он нема приступ ничему осим да направи налог.

2.2. ПРИЈАВЉЕНИ КОРИСНИК (СЛУШАЛАЦ)

Слушалац има приступ свим песмама, може да их претражи и послуша. Може да направи плејлисту и да дода било коју песму у њу. Када послуша песму број слушања се повећа за један у бази.

2.3. АДМИНИСТРАТОР

Администратор може да додаје песме у базу података да упише њено име, име аутора, жанр. Он исто има приступ свему што обичан корисник има.

2.4. МАТРИЦА ПРАВА ПРИСТУПА

Функционалност	Гост	Слушалац	Администратор
Регистрација	✓	✗	✗
Логовање	✓	✗	✗
Претрага песама	✗	✓	✓
Слушање музике	✗	✓	✓
Креирање плејлисти	✗	✓	✓
Додавање нових песама	✗	✗	✓

3. КРАТАК ОПИС КОРИШЋЕНИХ ТЕХНОЛОГИЈА

У развоју веб апликације за стримовање и репродукцију музике коришћен је скуп веб технологија које омућућавају стабилност, брзину и интерактивност корисничког интерфејса.

3.1. FRONTEND: КОРИСНИЧКИ ИНТЕРФЕЈС

- **HTML5:** Коришћен је за структурно дефинисање страница. Највећи значај има `<audio>` таг који омућућава нативно емитовање музике у претраживачу без потребе за екстерним додацима.
- **CSS:** Употребљен за визуелно обликовање апликације.
- **JavaScript & AJAX:** Кључна технологија за музичку платоформу. **AJAX** се користи за повећавање прегледа на песмама.

3.2. BACKEND: СЕРВЕРСКА ЛОГИКА

- **PHP:** Главни серверски језик који обрађује све захтеве. PHP проверава идентитет корисника, одговоран је за комуникацију са базом података, као и за манипулацију фајлова када се додаје песма на сервер.

3.3. БАЗА ПОДАТАКА

- **MySQL:** Релациона база података у којој се чувају сви кључни подаци. Структура базе је оптимизована тако да брзо враћа резултате претраге, чува hash-оване лозинке корисника.

3.4. РАЗВОЈНИ АЛАТИ (ENVIRONMENT)

- **XAMPP:** Локални серверски пакет који садржи Apache сервер и MySQL базу. Коришћен је као развојно окружење за тестирање апликације при развоју.

- **Visual Studio Code (VS Code):** Примарни уређивач кода. Коришћени су додаци за PHP IntelliSense и проверу синтаксе како би се смањила могућност грешке у коду.
- **phpMyAdmin:** Алат коришћен за визуелно моделовање базе података и извршавање тестних SQL упита.

Технологија	Логотип
HTML5	
CSS3	
JavaScript	
PHP	

4. ЛОГОТИП

Поред програмског дела, посебан акценат постављен је на визуелни идентитет апликације. За потребе пројекта креиран је логотип и помоћу алата **Adobe Photoshop**.

Процес је обухватио:

- Креирање векторских облика
- Обрада и оптимизација
- Колорна палета

4.1. ЛОГОТИП



5. МОДЕЛОВАЊЕ БАЗЕ ПОДАТАКА

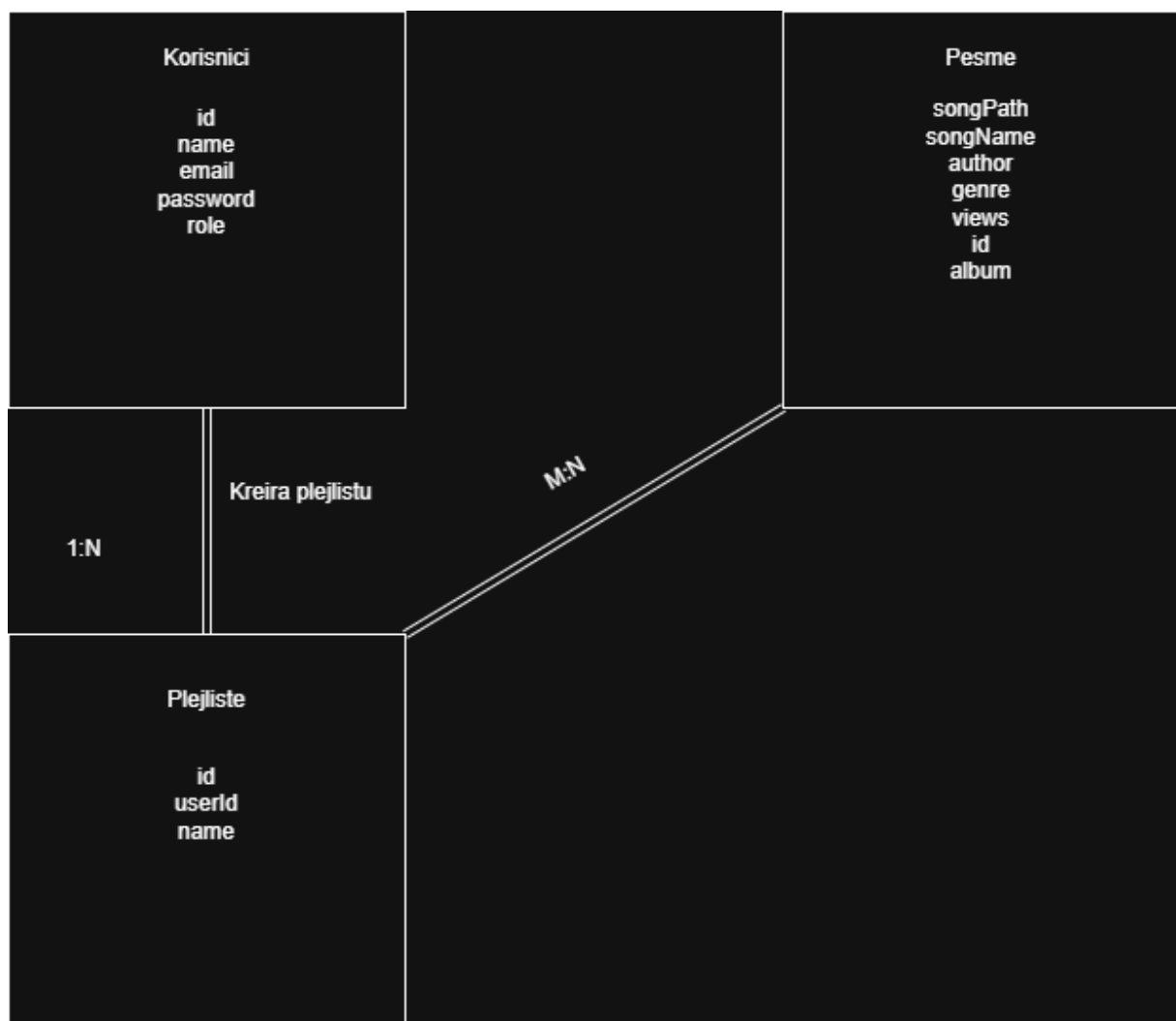
5.1. КОНЦЕПТУАЛНИ МОДЕЛ

Овај модел приказује шта ће се чувати у бази, фокусирајући се на ентитете и њихове међусобне односе без улажења у детаље.

Идентификовани ентитети:

1. **Корисници** – особе које користе систем.
2. **Песме** – песме које додаје админ.
3. **Плејлисте** – колекција песама
4. **Песме у плејлисти** – везна табела која повезује песме са плејлистом

ER diagram:



5.2. ЛОГИЧКИ МОДЕЛ

Логички модел дефинише структуру података (табеле, колоне, примарне и стране кључеве), али је и даље независан од конкретног софтвера који ће се користити. Овде се одређују типови података и правила, чиме се поставља прецизан нацрт за саму имплементацију.

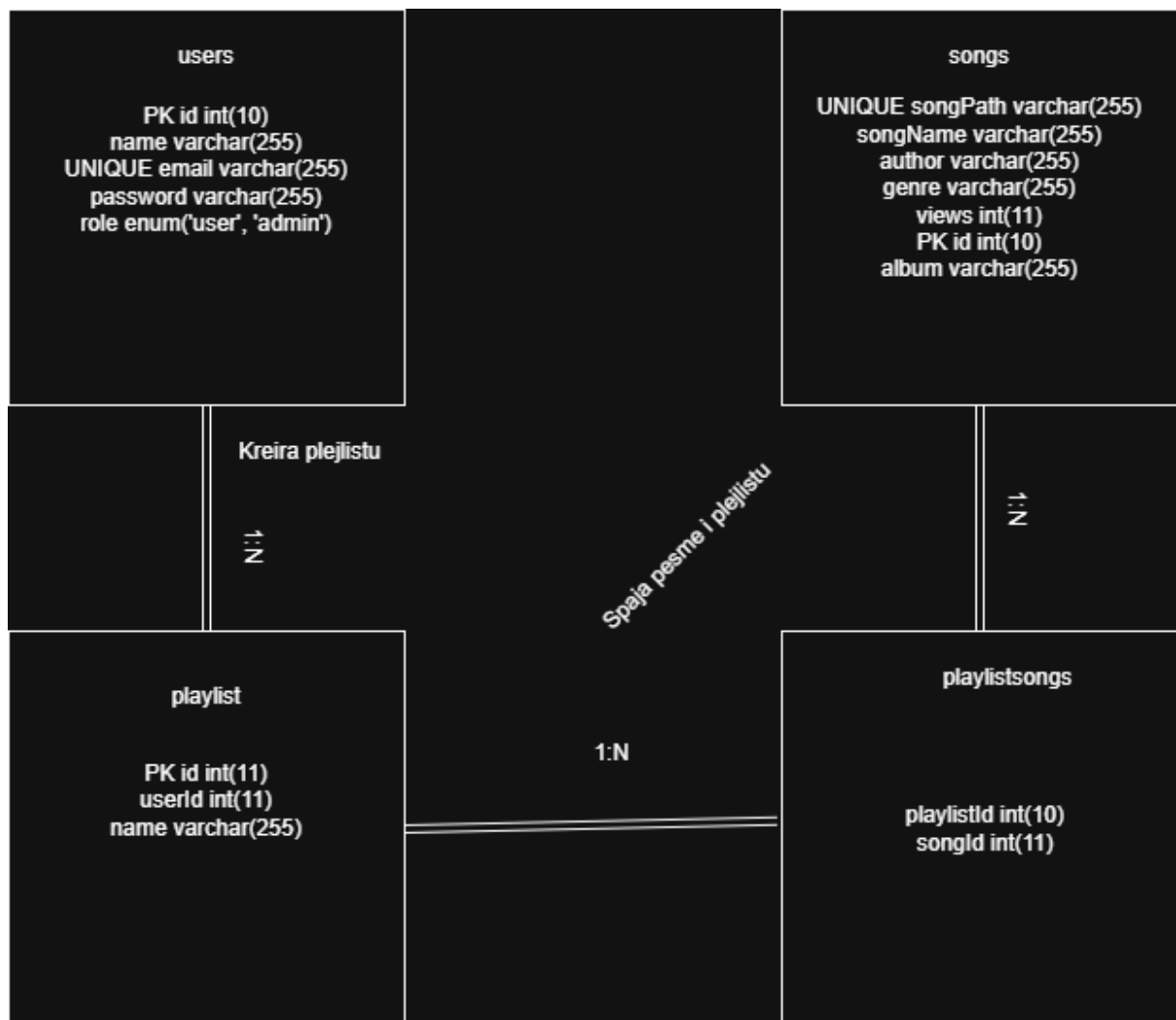
ЕР дијаграм:



5.3. ФИЗИЧКИ МОДЕЛ

Овај модел описује како се подаци физички складиште на диску у одређеном систему. Он обухвата специфичне SQL команде за креирање табела, дефинисање индекса ради брже претраге и управљање перформанса самог сервера.

ЕР дијаграм:



Спецификација табела:

playlist

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	id	int(11)			No	None		AUTO_INCREMENT
☐	2	userId	int(11)			No	None		
☐	3	name	varchar(255)	utf8mb4_general_ci		No	None		

playlistsongs

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	playlistId	int(11)			No	None		
☐	2	songId	int(11)			No	None		

songs

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	songPath	varchar(255)	utf8mb4_general_ci		No	None		
☐	2	songName	varchar(255)	utf8mb4_general_ci		No	None		
☐	3	author	varchar(255)	utf8mb4_general_ci		No	None		
☐	4	genre	varchar(255)	utf8mb4_general_ci		No	None		
☐	5	views	int(11)		UNSIGNED	No	0		
☐	6	id	int(10)		UNSIGNED	No	None		AUTO_INCREMENT
☐	7	album	varchar(255)	utf8mb4_general_ci		No	None		

users

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	id	int(10)		UNSIGNED	No	None		AUTO_INCREMENT
☐	2	name	varchar(255)	utf8mb4_general_ci		No	None		
☐	3	email	varchar(255)	utf8mb4_general_ci		No	None		
☐	4	password	varchar(255)	utf8mb4_general_ci		No	None		
☐	5	role	enum('user', 'admin')	utf8mb4_general_ci		No	None		

SQL DDL скрипта:

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Host: 127.0.0.1
-- Generation Time: Mar 03, 2026 at 02:34 PM
-- Server version: 10.4.32-MariaDB
-- PHP Version: 8.0.30

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Database: `users_db`
--

--
-- Table structure for table `playlist`
--

DROP TABLE IF EXISTS `playlist`;
CREATE TABLE `playlist` (
  `id` int(11) NOT NULL,
  `userId` int(11) NOT NULL,
  `name` varchar(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Table structure for table `playlistsongs`
--

DROP TABLE IF EXISTS `playlistsongs`;
CREATE TABLE `playlistsongs` (
```

```
`playlistId` int(11) NOT NULL,  
`songId` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
-----  
  
--  
-- Table structure for table `songs`  
--  
  
DROP TABLE IF EXISTS `songs`;  
CREATE TABLE `songs` (  
  `songPath` varchar(255) NOT NULL,  
  `songName` varchar(255) NOT NULL,  
  `author` varchar(255) NOT NULL,  
  `genre` varchar(255) NOT NULL,  
  `views` int(11) UNSIGNED NOT NULL DEFAULT 0,  
  `id` int(10) UNSIGNED NOT NULL,  
  `album` varchar(255) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
--  
-- Table structure for table `users`  
--  
  
DROP TABLE IF EXISTS `users`;  
CREATE TABLE `users` (  
  `id` int(10) UNSIGNED NOT NULL,  
  `name` varchar(255) NOT NULL,  
  `email` varchar(255) NOT NULL,  
  `password` varchar(255) NOT NULL,  
  `role` enum('user','admin') NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
--  
-- Indexes for table `playlist`  
--  
ALTER TABLE `playlist`  
  ADD PRIMARY KEY (`id`);  
  
--  
-- Indexes for table `songs`  
--  
ALTER TABLE `songs`  
  ADD PRIMARY KEY (`id`),  
  ADD UNIQUE KEY `song_Path` (`songPath`);
```

```
--  
-- Indexes for table `users`  
--  
ALTER TABLE `users`  
  ADD PRIMARY KEY (`id`),  
  ADD UNIQUE KEY `email_unique` (`email`);  
  
--  
-- AUTO_INCREMENT for dumped tables  
--  
  
--  
-- AUTO_INCREMENT for table `playlist`  
--  
ALTER TABLE `playlist`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;  
  
--  
-- AUTO_INCREMENT for table `songs`  
--  
ALTER TABLE `songs`  
  MODIFY `id` int(10) UNSIGNED NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=77;  
  
--  
-- AUTO_INCREMENT for table `users`  
--  
ALTER TABLE `users`  
  MODIFY `id` int(10) UNSIGNED NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;  
COMMIT;  
  
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;  
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;  
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```


6. ПРИКАЗ И ОПИС ДЕЛОВА КОДА

Овај део кода обезбеђује сигуран прекид корисничке сесије.

`session_start()`; Ова функција је неопходна на почетку сваке скрипте како би препознао постојећу сесију корисника и могао да манипулише подацима о њој.

`if (isset($_SESSION['email']))` Проверава постојање сесије, да ли је корисник пријављен. Ово представља основни механизам заштите приватних страница.

`session_unset()`; Ова команда брише све податке који су сачувани у сесији.

`session_destroy()`; Ова функција потпуно уништава сесију на серверу.

`header("Location: index.php")`; Ова функција се користи за преусмеравање на другу страницу. На пример при одјави шаље га назад на логин страницу. (`index.php`)

`logout.php`

```
<?php

session_start();
if (!isset($_SESSION['email'])) {
    header("Location: index.php");
    exit();
}
session_unset();
session_destroy();
header("Location: index.php");
exit();

?>
```

Ово је део кода одговоран за иницијализацију комуникације са сервером базе података. Искоришћена је MySQLi класа у објектно-оријентисаном стилу.

Дефинишу се 4 параметара, инстанцира се нови објекат који узима ове параметре као аргументе. После тога проверава грешке.

config.php

```
<?php

$host = "localhost";
$user = "root";
$password = "";
$database = "users_db";

$conn = new mysqli($host, $user, $password, $database);

if($conn->connect_error) {
    die("Connection failed: ". $conn->connect_error);
}

?>
```

Он се користи сваки пут када се интерактује са базом података.

```
require_once 'config.php';
```

Мора да има ову линију кода како би се користили његове промењиве.

deletePlaylist.php

```
$stmt1 = $conn->prepare("DELETE FROM playlistsongs WHERE playlistId = ?");
$stmt1->bind_param("i", $playlistId);
$stmt1->execute();

$stmt2 = $conn->prepare("DELETE FROM playlist WHERE id = ?");
$stmt2->bind_param("i", $playlistId);
$stmt2->execute();

$conn->commit();
```

При регистрацији користи се функција `password_hash`. Он чува hash-овану шифру корисника у бази података.

`login_register.php`

```
if (isset($_POST['register'])) {
    $name = $_POST['name'];
    $email = $_POST['email'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
    $role = $_POST['role'];

    $checkEmail = $conn->query("SELECT email FROM users WHERE email = '$email'");

    if ($checkEmail->num_rows > 0) {
        $_SESSION['register_error'] = 'Email is already registred';
        $_SESSION['active_form'] = 'register';
    } else {
        $conn->query("INSERT INTO users (name, email, password, role)
VALUES ('$name', '$email', '$password', '$role')");
    }

    header("Location: index.php");
    exit();
}
```

Лозинке се никада не чувају у чистом тексту, већ се пре уписа у базу врши њихово криптовање ради безбедности корисника.

AJAX Fetch за ажурирање броја слушања на песни. Коришћењем ове технологије, број слушања песме се ажурира у бази података у позадини, без прекидања музичког плејера и освежавања странице.

`player.js`

```
audioPlayer.addEventListener("playing", function() {
    if(updateView !== songId) {
        fetch(`updateViews.php?id=${songId}`);
        updateView = songId;
    }
});
```

Код за креирање MySQL упита за претраживање песама по имену, аутору и жанру.

search.php

```
$query = "SELECT * FROM songs WHERE 1=1";
if(isset($_POST['songName'])) {
    $songName = $_POST['songName'];
    $songName = mysqli_real_escape_string($conn, $songName);
    $query .= " AND songName LIKE '%$songName%'";
}
if(isset($_POST['author'])) {
    $author = $_POST['author'];
    $author = mysqli_real_escape_string($conn, $author);
    $query .= " AND author LIKE '%$author%'";
}
if(isset($_POST['genre'])) {
    $genre = $_POST['genre'];
    $genre = mysqli_real_escape_string($conn, $genre);
    $query .= " AND genre LIKE '%$genre%'";
}
$result = mysqli_query($conn, $query);
```

7. ЗАКЉУЧАК

Реализација веб система за стримовање музике успешно је објединила кључне аспекте савременог веб програмирања: рад са базама података, серверску логику у PHP језику и динамички кориснички интерфејс.

Највећи развојни изазов представљала је сама почетна фаза пројекта. Док се није пробио лед, напредак је био спорији и захтевао је интензивно коришћење литературе и онлајн докуменације. Међутим, након прва три дана, када су постављене основе система и када је савладан механизам комуникације између PHP-а и MySQL-а, процес израде је постао знатно бржи и ефикаснији.

Даљи рад на функционалностима, попут управљања плејлистама и имплементације AJAX позива, текао је природно и без велике потребе за додатним изворима, јер је логика система постала јасна. Овај пројекат ми је помогао да стекнем рутину у писању чистијег кода и да боље разумем како подаци "путују" од корисничког клика до базе података и назад.

Иако систем сада испуњава све техничке захтеве, ово искуство ми је показало да је најважнији део сваког пројекта управо то почетно савладавање препрека, након чега се фокус може у потпуности пребацити на оптимизацију и фино подешавање корисничког искуства.

Иако је апликација скроз функционална и испуњава све захтеве дефинисане упутством, постоји простор за даља унапређења. Будуће верзије апликације могле би да укључе имплементацију система за оцењивање песама, систем за ред песама као и коришћење модерних CSS радних оквира као што је Tailwind за модернији изглед интерфејса.

8. ЛИТЕРАТУРА

- [1.] MDN HTML документација [HTML: HyperText Markup Language | MDN](#)
- [2.] MDN CSS документација [CSS: Cascading Style Sheets | MDN](#)
- [3.] MDN JavaScript документација [JavaScript | MDN](#)
- [4.] MySQL документација [MySQL :: MySQL Documentation](#)
- [5.] PHP документација [PHP: Documentation](#)
- [6.] XAMPP [XAMPP Installers and Downloads for Apache Friends](#)
- [7.] Visual Studio Code [Download Visual Studio Code - Mac, Linux, Windows](#)
- [8.] Предавања из предмета програмирање веб апликација.
- [9.] Предавања из предмета базе података.
- [10.] Stack Overflow [Newest Questions - Stack Overflow](#)
- [11.] Luke Welling, Laura Thomson - PHP i MySQL:razvoj aplikacija za Web (+ CD)
, било које издање (Mikro knjiga)
- [12.] Vida i Jovan Popović - jQuery i napredne web tehnologije
- [13.] Josh Hill i James A. Brannan - Briljantno HTML5 i CSS3 (CET)
- [14.] draw.io [draw.io](#)